

Demo-Based Round-Win Inference for Localized Learning

A Multi-Map Machine Learning Model for CS2 Win Probability Prediction

Jonathan Lin
Arizona State University
1222211003
jjlin3@asu.edu

Prabhu Pugalenth
Arizona State University
1237213174
ppugale1@asu.edu

Chaz Wyllie
Arizona State University
1229388955
chazwyllie@gmail.com

ABSTRACT

DRILL (Demo-Based Round-Win Inference for Localized Learning) is a system designed to estimate Counter-Strike round outcomes from a single game state using machine learning. Building prior work such as Xenopoulos et al. (2020), which formalized context-aware action valuation and round win-probability modeling, our approach extracts structured features from parsed .dem files and applies supervised learning to predict round outcomes in real time. In this work, we extend these methods to Counter-Strike 2 (CS2) and construct a unified multi-map dataset spanning the full competitive map pool. Using gradient-boosted models, particularly CatBoost, we evaluate win-probability prediction across all maps and demonstrate that the model produces accurate and well-calibrated probabilities. These results indicate that DRILL provides a robust foundation for analysis, casting support tools, and future player-training or coaching applications.

CCS CONCEPTS

- Computing methodologies → Machine learning; Supervised learning; Gradient boosting methods.
- Applied computing → Computer games, Decision analysis.

KEYWORDS

CT – Counter Terrorist, T – Terrorist, Logloss – probability accuracy, ROC-AUC – ranking quality, Brier Score – squared error in probabilities, ECE – expected calibration error.

Introduction

Competitive eSports such as Counter-Strike generate copious quantities of structured match data, including player positions, health, weapon loadouts, utility usage, and round outcomes. Prior research shows that this data can be used to estimate contextual win probability during rounds (Xenopoulos et al., 2020) [1]. However, many existing tools display only raw statistics rather than predictive, interpretable round-state probabilities. The DRILL project (Demo-Based Round-Win Inference for Localized Learning) investigates round win probability prediction using a machine learning model trained on parsed demo data. Given the current state of a round, such as alive players, equipment value, bomb status, and positional advantage; the model outputs the probability that either team will win. By following established methodologies in esports analytics research and adapting them to

our dataset, our goal is to reproduce and extend prior win-probability work using modern CS2 data. In a real-time setting, these probabilities can power overlays and dashboards that help analysts, casters, and competitive observers understand momentum shifts, evaluate decision-making, and even better anticipate which team is favored to win the overall match as round outcomes accumulate. By following established methodologies in esports analytics research and adapting them to our dataset, our goal is to reproduce and extend prior win-probability work using modern CS2 data.

Related Work

The DRILL project builds on a range of research in esports analytics, performance modeling, and coaching frameworks. Each paper contributes to a specific layer of DRILL, ranging from data collection to post-game modeling, evaluation, and prediction.

Xenopoulos et al. (2020) [1] and **Xenopoulos et al. (2022) [2]** provide the data infrastructure essential for DRILL’s analytic pipeline. The 2020 study introduces Win Probability Added (WPA) for action valuation [1], while the 2022 study delivers the dataset along with awpy toolkit [2]. Together, they both define the groundwork for ingesting and structuring CS: GO demo files.

Marshall et al. (2022) [3] demonstrates how telemetry features enable real-time event predictions via deep-learning models, and **Rulff et al. (2022) [4]** introduces scalable visualization/indexing methods for state-level analysis. Both papers contribute significantly to DRILL’s modeling and interface design, from displaying how in-game data is processed, structured, and visualized for feedback analysis.

Bialecki et al. (2023) [5] and **Sharpe et al. (2022) [6]** introduce the foundation of human-performance and conceptual frameworks, connecting data analytics to player development. Their insights into the physical, cognitive, and psychological components of high-level esports performance and metrics for analyzing directly aligning with DRILL’s task of converting computer data into human-comprehensible information.

Bahrololoomi et al. (2023) [8] and **Bubna et al. (2023) [7]** both further enhance the relationship between evaluation and coaching systems based on esports performance. Bahrololoomi et al. (2023) consists of machine learning performance metrics used to evaluate a player’s performance and predict outcomes [8], contributing to

DRILL’s evaluation suite. Bubna et al. (2023) consists of constraints used as a framework for coaching, which provides insights into DRILL’s recommendation model, further improving feedback/support that are evidence-based and supports the theory of skill acquisition [7].

Together, these studies form the technical foundation for the win-probability component of DRILL. Prior work provides insights into action valuation (Xenopoulos et al., 2020) [1], large-scale esports datasets (Xenopoulos et al., 2022) [2], event prediction using telemetry (Marshall et al., 2022) [3], and scalable state representations (Rulff et al., 2022) [4]. Additional research on player-performance metrics (Bahrololoomi et al., 2023) [7] and standardized measurement frameworks (Sharpe et al., 2022) [6] supports our model evaluation design. Collectively, these studies guide the construction of our round win-probability predictor by informing how features should be engineered, modeled, and evaluated.

Data

In Counter-Strike, every online match runs on a central game server that continuously receives input commands from the clients and updates the shared game state. This communication happens at a fixed temporal resolution known as the tick rate: a tick is a single discrete update step of the server (typically 64 or 128 times per second), during which the server processes player inputs, resolves physics and damage, updates entities such as grenades and the bomb, and then sends the new state back to all connected players. When demo recording is enabled, Valve’s engine logs this evolving sequence of game states and events into a single replay file with the .dem extension. Conceptually, the .dem file is a chronological record of what the server “saw” at each tick: which players were alive, where they were positioned, what actions they performed, and how the round progressed from start to finish.

Internally, a parsed .dem file can be viewed as a collection of structured tables derived from this tick-by-tick log. For each tick and for each player, the engine records information such as match and round identifiers, map name, server tick number, and wall-clock time since the round started, as well as player-level fields like team (CT or T), health, armor, helmet flag, active weapon, money, and three-dimensional position together with view angles (yaw and pitch). Additional event-level records capture actions such as weapon fires, grenade throws, player-hurt and player-death events, bomb plants and defuses, and score updates. Together, these fields describe both the instantaneous state of the game and the sequence of actions that transformed one state into the next; our preprocessing pipeline reads the .dem file, reconstructs the underlying trajectories, and then aggregates them into a more compact representation suitable for machine learning.

The project relies on three primary datasets: (1) the public Esports Trajectories and Actions (ESTA) dataset by Xenopoulos and Silva (2022), (2) a newly curated professional CS2 dataset collected from HLTV and FACEIT matches, and (3) personal CS2 demos provided by team members for calibration and evaluation.

Together, these sets form the foundation of DRILL’s learning and testing framework. ESTA contains approximately 1,558 professional CS:GO replays, representing over 41,000 rounds and millions of logged events, while the pro CS2 and personal demo sets extend this corpus to modern game versions and individual play styles. All three sources are parsed through the same pipeline to produce a single, consistent tabular representation of round states.

secondsSincePhaseStart	bombPlanted	bombsite	totalSmokes	totalFires	ctAlive	ctHp	ctArmor	ctHelmet	ctEq	ctUtility	ctEqValStart	ctBombZone	defuser
38.18110236	FALSE		0	0	4	281	188	0	4250	5	5100	0	
31.00767402	FALSE		2	0	5	473	498	5	19700	7	20000	0	
16.03149606	FALSE		2	1	5	500	486	5	24400	13	23800	0	
11.16535433	FALSE		2	2	5	500	486	5	28950	13	28950	0	
48.51181102	FALSE		2	0	5	499	482	4	30700	6	30700	0	
59.69291339	FALSE		2	0	2	200	200	2	11000	1	30050	0	
30.92913386	FALSE		0	0	5	500	496	5	29950	10	30550	0	
26.19685039	FALSE		2	0	5	500	484	5	30450	11	29050	0	
60.25984252	FALSE		1	1	5	481	482	5	30350	4	30950	0	
18.56692913	FALSE		2	0	4	366	380	4	25050	13	30950	0	
73.50393701	FALSE		0	0	5	399	461	5	28750	7	31150	0	
20.22834646	FALSE		3	0	3	300	285	3	17600	7	31150	0	
15.01574803	FALSE		6	3	5	500	500	4	31100	10	31100	0	
1.913385827	TRUE	B	0	0	3	202	290	3	19150	2	31350	0	
59.09448819	FALSE		0	0	3	239	291	3	16400	5	32650	0	
10.64566929	FALSE		0	0	5	500	400	0	4400	1	4000	0	
6.078740157	FALSE		0	0	5	500	500	5	21150	11	21150	0	
88.2519685	FALSE		0	0	2	200	200	0	5050	0	6950	0	
15.86614173	FALSE		2	2	5	494	497	3	9800	2	9800	0	
40.99212598	FALSE		1	0	4	375	400	4	21700	6	24850	0	
39.79527559	FALSE		1	0	5	500	500	5	26200	7	26550	0	
43.54330709	FALSE		2	0	4	400	390	4	23000	6	28700	0	
45.48818898	FALSE		1	0	4	315	376	4	20700	6	27700	0	
27.19685039	FALSE		1	1	3	298	283	3	18200	4	29350	0	
16.515625	FALSE		0	0	5	500	500	0	4250	0	4250	0	

Figure 1: The tabular dataset, containing all the features we extracted from ESTA, pro matches, and our own games.

Each row corresponds to a single sampled game state within a round, and each column encodes a specific aspect of the current situation. Concretely, each feature vector includes:

1. **Seconds since round started** - continuous time into the active phase of the round.
2. **Bomb planted flag** - whether the bomb is currently planted at this state.
3. **Bombsite identifier** - which site (A/B) the bomb is planted on, if applicable; otherwise “none.”
4. **Total number of smoke grenades used so far** in the round.
5. **Total number of Molotov/incendiary grenades used so far** in the round.
6. **Team state for CT and T** – aggregate health points (HP), armor, number of helmets, weapons/equipment value, utility (grenades), and current balance for both teams.
7. **Number of defuse kits** purchased by the CT side.
8. **Bomb carrier status** – whether a Terrorist player is currently holding the bomb.

These features jointly describe the tactical and economic state of both teams at a given moment and form the input to our round-win probability models.

Methods

The .dem files are first parsed using Python demo-parsing libraries such as Demoparser2 [8] and AWPY [9], which extend the low-level Valve demo format into higher-level Python objects and tables. At a prominent level, these parsers iterate over the demo tick by tick, decode the network messages sent by the game server, and expose them as structured events and state snapshots (e.g., player positions, damage events, grenade throws, bomb plants, round starts/ends). For each demo, we use these tools to reconstruct the full sequence of rounds and per-tick game states and then serialize the intermediate results as both pickle files (for fast re-loading in Python) and CSV files (for inspection and downstream tabular processing). Once the raw events have been exported, we perform a cleaning stage in which we check for parsing errors, missing values, and obvious abnormalities (e.g., negative health, impossible money values). We then normalize numeric fields where appropriate, convert Boolean flags to binary 0/1 indicators, remove duplicated states, and ensure that all maps and rounds share a consistent schema.

From these cleaned tables, we transform each sampled game state into a compact feature vector following the approach of Xenopoulos et al. (2020) [1]. Instead of using raw tick-level data directly, we aggregate per-player and per-event information into round-state features that summarize the tactical situation at a given moment. In particular, we focus on the eight features described in the Data section: seconds since the round started, a bomb-planted flag, a bombsite identifier, cumulative counts of smokes and Molotov/incendiary grenades used, aggregated team statistics for CT and T (health, armor, helmets, equipment value, utility, and balance), the number of CT defuse kits, and a bomb-carrier flag. These features fall into categories commonly used in esports prediction research—alive players and total health, equipment and utility, bomb status, and economic state—and are designed to reflect the real-time strategic conditions that influence round outcomes (Xenopoulos et al., 2020 [1]; Rulff et al., 2022 [4]). Each feature vector is paired with a binary target indicating whether the CT team eventually wins the round (1) or loses (0).

To model round win probability, we treat this as a supervised learning problem and train gradient-boosted decision tree models on the resulting tabular dataset. We compare two widely used libraries: XGBoost and CatBoost. XGBoost implements gradient boosting over decision trees with support for regularization, subsampling and efficient handling of large tabular datasets, but it requires us to manually encode all categorical variables. In our case, categorical fields such as bombsite (and, in the full multi-map setting, map name) must be factorized into integer codes or one-hot encodings before training, and Boolean flags must be converted to 0/1. CatBoost, by contrast, is specifically designed for heterogeneous tabular data and can consume categorical features natively; we simply specify the indices of categorical columns, and CatBoost internally learns target statistics and performs ordered boosting to reduce target leakage. Because our dataset mixes numeric features (health, armor, counts, money) with a small

number of categorical flags (bomb site, bomb planted, bomb carrier), we expect CatBoost to handle these interactions more naturally and to produce slightly better calibrated probabilities than XGBoost, with less manual feature engineering.

For training, we split the combined dataset into training, validation, and test partitions (e.g., 70%/10%/20%), stratified by the round outcome label so that the CT/T win-rate distribution is similar across splits. Both XGBoost and CatBoost are trained on the same training set and tuned on the same validation set to ensure a fair comparison. We perform light hyperparameter tuning by varying the number of trees (iterations), maximum tree depth, learning rate, and subsampling parameters, selecting configurations that reduce validation loss without obvious overfitting. The final models are then evaluated on the held-out test set.

Evaluation Metrics

Following prior work on win-probability modeling, we evaluate our models using logarithmic loss (logloss), Brier score, area under the ROC curve (ROC-AUC), and Expected Calibration Error (ECE). These metrics jointly assess both *discrimination* (whether the model ranks winning rounds above losing rounds) and *calibration* (whether a predicted probability of, say, 0.70 corresponds to a 70% - win rate in practice).

Let $y_i \in \{0,1\}$ denote the true outcome for round i (1 if CT wins, 0 otherwise) and let $p_i \in [0,1]$ be the predicted probability that CT wins that round.

Logarithmic loss (Logloss)

Logloss measures how well the predicted probabilities match the true labels in terms of negative log-likelihood. It penalizes confident but wrong predictions very heavily. For N examples, logloss is defined as

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

A lower logloss indicates better probabilistic predictions; perfectly predicted probabilities would yield a logloss of 0.

Brier score

The Brier score is the mean squared error between the predicted probabilities and the true binary outcomes. It directly measures how close each probability is to the actual result:

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2$$

Here again, lower values are better. Compared to logloss, the Brier score is less sensitive to rare, extremely misclassified cases, and can be interpreted as an overall “probability MSE.”

ROC-AUC

ROC-AUC measures the model’s ability to rank winning rounds above losing rounds, independent of any decision threshold.

Formally, it can be interpreted as the probability that a randomly chosen positive example (CT win) receives a higher score than a randomly chosen negative example (CT loss). One common formulation is

$$\text{AUC} = \frac{1}{N_+ N_-} \sum_{i: y_i=1} \sum_{j: y_j=0} [\mathbb{I}(p_i > p_j) + \frac{1}{2} \mathbb{I}(p_i = p_j)]$$

where N_+ and N_- are the numbers of positive and negative rounds respectively, and $\mathbb{I}(\cdot)$ is the indicator function. For ROC-AUC, higher values are better; an AUC of 0.5 corresponds to random guessing, while an AUC of 1.0 indicates perfect ranking.

Expected Calibration Error (ECE)

While logloss and Brier score measure overall probabilistic accuracy, ECE focuses specifically on calibration, whether predicted probabilities correspond to actual empirical frequencies. A perfectly calibrated model would satisfy conditions like “among all rounds where the model predicts 0.70, the CT side actually wins about 70% of the time.” To compute ECE, we partition the predictions into M confidence bins (e.g., 10 bins covering $[0,0.1), [0.1,0.2), \dots, [0.9,1.0]$).

For each bin B_m , we compute the average predicted probability $\text{conf}(B_m)$ and the empirical accuracy $\text{acc}(B_m)$:

$$\text{conf}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} p_i, \text{acc}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} y_i$$

The Expected Calibration Error is then defined as the weighted average absolute gap between confidence and accuracy across bins:

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)|$$

An ECE of 0 indicates perfect calibration; larger values indicate that the model’s probabilities are systematically overconfident or underconfident in certain ranges. In our setting, ECE is the primary metric of interest, because our goal is not only to identify the favored team but also to provide numerically meaningful win probabilities that can be trusted by analysts and observers in real-time. Thus, for ECE, lower values are better.

In summary, for logloss, Brier score, and ECE, lower is better, reflecting fewer and smaller errors in the predicted probabilities, while for ROC-AUC, higher is better, reflecting stronger ranking performance. Using this set of metrics allows us to evaluate both how well the models discriminate between winning and losing rounds and how well-calibrated their probability estimates are for use in live esports analytics.

Results

On the held-out test set, both models achieve strong performance, but CatBoost consistently matches or slightly improves upon XGBoost across all four-evaluation metrics.

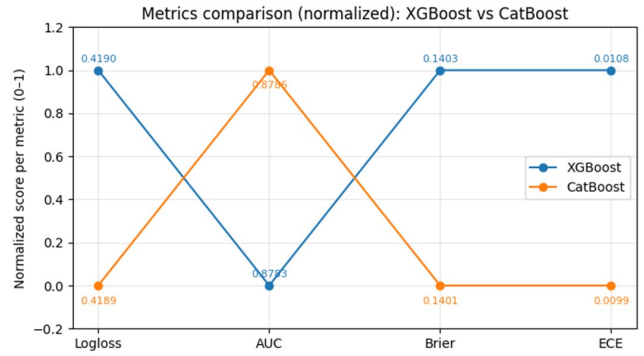


Figure 2 Normalized metrics comparison

The Figure 2 compares model performance across four key metrics Logloss, AUC, Brier score, and Expected Calibration Error (ECE), with all values normalized to a 0-1 range for visual comparability. (The raw, unscaled values are reported in Table 1.)

Metrics	XG Boost	CatBoost
LogLoss	0.4190	0.4189
ROC-AUC	0.8783	0.8786
Brier Score	0.1403	0.1401
ECE	0.0108	0.0099

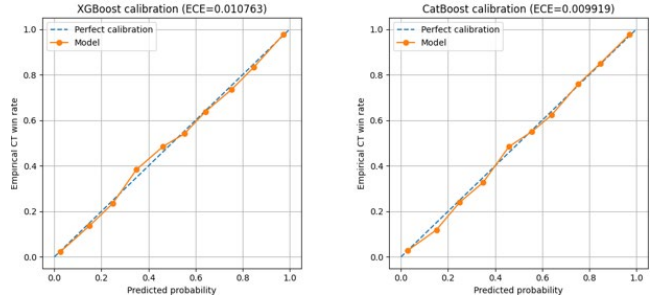


Figure 3 Results of Expected Calibration Error (ECE)

The Figure 3 plots calibration curves showing how closely each model’s predicted probabilities match the actual empirical frequency of CT round wins. Each panel shows a reliability diagram where the x-axis is the predicted CT win probability (grouped into bins) and the y-axis is the empirical CT win rate within each bin. The dashed diagonal represents perfect calibration, where predicted probabilities exactly match observed frequencies. The orange points for both models lie close to this line, indicating well-calibrated predictions; CatBoost tracks the diagonal slightly more closely and achieves a lower ECE (0.0099 vs. 0.0108), meaning its probability estimates are marginally better aligned with actual round outcomes.

Conclusion

Our unified multi-map DRILL model shows that CS2 round win probability can be accurately predicted from a single game state using gradient-boosted trees. Compared to XGBoost, CatBoost reduces logloss by about 0.02%, reduces Brier score by about 0.14%, and improves calibration error (ECE) by roughly 8%. AUC also increases slightly, indicating a small but consistent gain in ranking quality. Feature-level analysis confirms that the model has learned intuitive patterns: across maps, states with more CT players alive and higher total CT health are assigned higher CT win probabilities, and predicted probabilities become more decisive as the round progresses and uncertainty resolves. Overall, both models perform well, but CatBoost consistently edges out XGBoost on all error metrics, with the most notable improvement in calibration, which is crucial for interpretable win-probability overlays.

Future Work

Future work will extend DRILL beyond round-level prediction toward player-level evaluation and coaching. Using the same .dem files, we plan to extract fine-grained positional and view-angle telemetry, per-tick player coordinates (x, y, z) and view angles (yaw, pitch) and link them to fight outcomes. This will allow us to identify systematic positional issues such as overexposed angles, poor spacing, or suboptimal peeks on specific maps and sites. On top of this telemetry, we aim to build an AI “coach” layer that analyzes a player’s demos and generates personalized feedback on both positioning and aim behavior, for example highlighting cases where crosshair placement is too low, reaction distances are too large, or pre-aim is consistently off by a few degrees. Finally, we envision wrapping this analysis in an interactive chatbot that can translate these findings into concrete training routines recommending specific workshop maps, scenario drills, and positional practice for the situations where a player struggles most. In this way, DRILL can evolve from a pure analytics model into a comprehensive, personalized training assistant for competitive FPS players.

REFERENCES

- [1] Xenopoulos, P., Doraiswamy, H., & Silva, C. (2020). Valuing Player Actions in Counter Strike: Global Offensive. 2020 IEEE International Conference on Big Data (Big Data), 1283– 1292.
- [2] Xenopoulos, P., Rulff, J., ... & Silva, C. (2022). ESTA: An Esports Trajectory and Action Dataset (awpy). arXiv:2212.04356.
- [3] Marshall, S., West, S., & Ghinea, G. (2022). Enabling Real-Time Prediction of In-game Deaths through Telemetry in CS:GO. FDG 2022.
- [4] Rulff, J., Xenopoulos, P., ... & Silva, C. (2022). ggViz: Accelerating Large-Scale Esports Game Analysis. Proc. ACM Hum.-Comput. Interact. 6, CHI PLAY, Article 238 (Oct 2022).
- [5] Bialecki, A., Xenopoulos, P., ... (2023). ESPORT: Electronic Sports Professionals— Observations and Reflections on Training (preprint).
- [6] Sharpe, B. T., Besombes, N., Welsh, M., & Birch, P. (2022). Indexing Esport Performance. Journal of Electronic Gaming and Esports, 1(1), 1–13.
- [7] Farnod Bahrololloomi, Fabio Klonowski, Sebastian Sauer, Robin Horst, and Ralf Dörner. 2023. E-Sports Player Performance Metrics for

predicting the outcome of League of Legends matches considering player roles. SN Computer Science 4, 3 (March 2023).

- [8] Laiho, E. *demoparser2: Demo parser for Counter-Strike 2* [Software]. GitHub repository, <https://github.com/LaihoE/demoparser>
- [9] Xenopoulos, P. *awpy: Python library to parse, analyze, and visualize Counter-Strike demo files* [Software]. GitHub repository, <https://github.com/pnxenopoulos/awpy>